

計算機視覺 期末報告

即時手部復健偵測與智慧醫療姿勢輔助系統

組別：第九組

組員：資工三 S1254006 許靜諭

資工三 S1254054 郭品辰

資工三 S1261115 鄭采霓

資工三 S1267013 方梓寧

日期：2025年12月31日

摘要

腕道症候群（Carpal Tunnel Syndrome, CTS）患者在術後或保守治療期間，常因居家復健缺乏專業指導與即時回饋，導致動作不標準及依從性低落，進而影響康復成效。針對此問題，本研究提出一套基於電腦視覺的即時手部復健姿勢輔助系統，旨在提供低成本且高精度的居家復健監測方案。

本系統採用創新的「雙軌分析機制（Dual-Track Analysis Mechanism）」。首先利用 YOLOv8 模型進行影像背景分割與去背，並透過 MediaPipe 框架提取高精度的手部骨架關鍵點；同時，並行計算基於凸包（Convex Hull）、實心度（Solidity）與缺陷點（Defects）的幾何特徵，以補強深度學習模型在處理相似手勢時的不足。此外，系統引入有限狀態機（Finite State Machine, FSM）邏輯，以確保復健動作在時序上的完整性與規範性。

實驗結果顯示，相較於基礎深度學習模型，在納入 z 軸空間資訊後，整體辨識準確率由 82.00% 顯著提升至 98.38%。特別是在「angry_fist（生氣的握拳）」與「fist（拇指在內的握拳）」等外型高度相似的動作上，誤判率大幅降低，顯示三維空間資訊對於細微動作差異的辨識具有關鍵影響。此外，實驗亦驗證導入幾何特徵增強機制的雙軌模型，同樣能有效提升動作辨識準確率，顯示結合 AI 推論與傳統電腦視覺特徵的混合式方法，在復健動作偵測上具備良好的強健性與應用潛力，為未來居家智慧醫療與遠距照護系統之發展奠定重要基礎。

目錄

摘要	1
目錄	2
第一章 緒論	4
第一節 研究背景	4
第二節 研究動機與問題	4
第二章 文獻回顧	5
第一節 手部辨識技術演進	5
第二節 現有手勢資料集之技術參考	5
第三章 研究方法	6
第一節 系統開發環境與函式庫	6
第二節 系統架構概述	6
第三節 復健動作定義與數據採集	7
第四節 影像前處理與關鍵點偵測	8
第五節 雙軌特徵分析機制	9
第六節 復健動作狀態機	10
第四章 實作	11
第一節 函示庫	11
第二節 程式碼	12
第五章 實驗結果與分析	17
第一節 實驗設置	17
第二節 模型效能比較	17
第三節 各類別辨識分析	19
第四節 混淆矩陣分析	20

第五節 訓練收斂過程	21
第六節 實作困難與技術突破	21
第六章 結論與未來展望	22
第一節 結論	22
第二節 未來展望	22
第七章 參考文獻	23

第一章 緒論

第一節 研究背景

腕道症候群（Carpal Tunnel Syndrome, CTS）是現代最常見的上肢周邊神經壓迫病變之一。根據 Atroshi 等人的研究指出，CTS 在一般大眾中的盛行率約為 3.8% 至 14.4% 不等，且女性發病率顯著高於男性 [1]。隨著現代工作型態轉變，長時間使用電腦與智慧型手機的族群日增，長期重複性的手腕動作導致 CTS 的發病風險顯著提升 [2]。

臨床上，CTS 的初期治療多採保守療法，其中「神經滑動運動」（Nerve Gliding Exercises）被廣泛證實能有效緩解正中神經的壓力並改善手部功能。物理治療師通常會建議患者居家進行持續性的復健運動，以避免手術介入 [3]。

第二節 研究動機與問題

儘管居家復健具有便利性與低成本優勢，但其實際成效往往受限於患者的「依從性」（Compliance）與「動作準確度」。研究顯示，居家復健計畫的患者依從性普遍低落，約僅有 35% 至 50% 的患者能完全遵照醫囑完成運動 [4]。造成依從性低落的主因包括：復健過程枯燥乏味、缺乏即時回饋、以及患者無法確定自身動作是否標準 [5]。

傳統的居家復健缺乏客觀的監測機制，物理治療師難以量化患者在家中的復健次數與品質。若患者以錯誤姿勢重複練習，不僅無法達到治療效果，甚至可能造成二次傷害。因此，如何利用自動化技術來輔助居家復健，提供即時的姿勢矯正與數據記錄，成為當前智慧醫療領域的重要研究課題 [6]。

近年來，電腦視覺（Computer Vision）與深度學習技術的突破，為上述問題提供了新的解決方案。相較於穿戴式裝置（如數據手套）昂貴且穿脫不便，基於影像辨識的非接觸式系統更適合居家推廣 [7]。本研究旨在開發一套基於 YOLO、MediaPipe、計算幾何與拓撲結構分析的即時手部復健偵測系統，以解決傳統居家復健缺乏監控與回饋之痛點。

第二章 文獻回顧

第一節 手部辨識技術演進

早期的手勢辨識多依賴傳統影像處理技術，如膚色檢測（Skin Color Detection）與邊緣特徵提取，容易受光影變化與背景雜訊干擾。近年來，隨著深度學習技術的突破，基於卷積神經網路（CNN）的方法已成為主流。

YOLO (You Only Look Once) 系列演算法因其單階段（One-stage）偵測的高效率，被廣泛應用於即時物件偵測任務中。最新的研究指出，YOLOv8 在輕量化與推論速度上取得了顯著平衡，適合應用於居家設備的邊緣運算環境 [8]。此外，Google 提出的 MediaPipe Hands 框架，透過輕量化的 3D 手部骨架模型，能從單張 RGB 影像中精確推論出 21 個關鍵點，解決了傳統方法難以處理的手部遮擋與複雜姿態問題 [9]。

本研究參考了上述技術，提出結合 YOLO 物件偵測與 MediaPipe 骨架特徵的整合方案，以提升復健動作偵測的強健性。

第二節 現有手勢資料集之技術參考

在建構本系統的手勢辨識模型前，本研究深入分析了目前公開資料集，如 HaGRID [11]、IPN Hand [12] 以及 Sign Language Digits [10]。這些資料集提供了手勢辨識在「資料標註策略（Annotation Strategy）」、「背景複雜度處理」以及「模型架構選擇」上的重要技術參考。例如，HaGRID 證明了在訓練資料中引入豐富的背景與光照變化，對於提升模型在非受控環境下的魯棒性至關重要。

然而，經檢視上述資料集，我們發現其收錄之手勢多為通用型指令（如停止、比讚）或手語手勢，缺乏針對「腕道症候群（CTS）」復健所需的特定醫療級動作（如肌腱滑動練習）。鑑於目前尚無針對此類專業復健手勢的開源資料集，本研究確立了自行建立數據集的必要性。我們參考了上述國際資料集的採集規範，並依據臨床物理治療指引錄製影像，以確保數據集能精確反映復健所需的生理特徵。

第三章 研究方法

第一節 系統開發環境與函式庫

本系統基於 Python 3.10 語言開發，並整合多個開源函式庫以實現影像處理、模型推論與使用者介面。主要使用的關鍵函式庫與版本如下：

1. OpenCV (opencv-python $\geq$ 4.8.0)：作為核心影像處理引擎，負責讀取攝影機串流 (Video Stream)、影像縮放、色彩空間轉換以及幾何圖形 (如凸包、輪廓) 的繪製與計算。
2. MediaPipe (mediapipe $\geq$ 0.10.21)：由 Google 開發的機器學習框架，本研究利用其 Hands 模組進行手部偵測。相較於傳統方法，MediaPipe 能在 CPU 上即時推論出 21 個 3D 手部關鍵點 (Landmarks)，為雙軌分析提供高精度的骨架數據。
3. TensorFlow (tensorflow $\geq$ 2.13.0)：用於建構與訓練自定義的深度神經網路 (MLP)。本系統利用 Keras API 搭建輕量化分類模型，處理正規化後的關鍵點座標數據。
4. NumPy (numpy $\geq$ 1.24.3)：負責所有矩陣運算與數據結構處理，包括將 MediaPipe 的座標點轉換為向量格式，以及計算手部幾何特徵 (如長寬比、歐氏距離)。
5. CVZone：用於優化即時影像的視覺化呈現 (UI Rendering)，包括動態文字顯示與半透明圖層疊加，提升使用者的互動體驗。

第二節 系統架構概述

本研究提出一套整合深度學習與幾何特徵分析的即時手部復健輔助系統。系統運作流程分為三個主要階段：(1) 影像前處理與感興趣區域 (Region of Interest, ROI) 提取、(2) 雙軌特徵分析 (Dual-Track Feature Analysis)、以及 (3) 復健動作狀態判讀。系統輸入為即時攝影機影像流，輸出則為復健動作的計數與姿勢正確性回饋。整體架構如圖 1 所示。

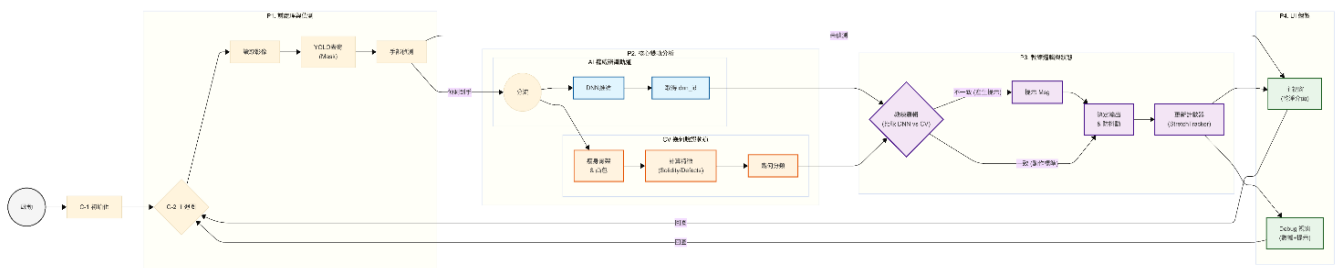


圖 1：系統流程圖

第三節 復健動作定義與數據採集

由於目前缺乏針對 CTS 復健的公開影像資料集，本研究自行建立了一套包含 8 種類別的專用手勢數據集。動作設計根據物理治療師丁奕 (Ding, 2021) 之臨床指導 [14]，本研究選用的肌腱滑動運動 (Tendon Gliding Exercises) 旨在解決腕道症候群 (CTS) 術後或保守治療中常見的肌腱沾黏問題。本研究選定的各動作之具體訓練目標與解剖學意義詳述如下表，包含以下 5 項標準臨床動作、2 項自定義功能性手勢，以及除上述 7 種手勢以外的其他狀態手勢。

中文手勢名稱	英文手勢名稱	動作特徵描述	生理訓練目標與效益
勾拳	Hook	保持手腕正中，掌指關節 (MCP) 伸直，彎曲近端 (PIP) 與遠端 (DIP) 指間關節。	促進深淺肌腱相對滑動。 此動作能產生最大的「屈指深肌 (FDP)」與「屈指淺肌 (FDS)」之間的相對滑動位移，防止兩條肌腱互相沾黏。
生氣的握拳	Angry Fist	保持手腕正中，拇指在四指之上，手指所有關節完全彎曲並用力握緊。	最大化肌腱滑動距離。 促進 FDP 與 FDS 肌腱相對於腱鞘 (Sheath) 與骨頭的最大滑動，同時強化握力。
鴨子	The Duck	彎曲掌指關節 (MCP)，同時維持指間關節 (IP) 伸直，形似鴨嘴。	強化手部核心肌群。 主要訓練手掌內部的細小肌群：蚓狀肌 (Lumbricals) 與 骨間肌 (Interossei)。這兩組肌群對手部精細動作至關重要。
直拳	Straight Fist	彎曲近端指間關節 (PIP)，維持遠端指間關節 (DIP) 伸直，指尖觸碰掌心。	促進淺肌滑動。 此動作能產生最大的「屈指淺肌 (FDS)」相對於腱鞘與骨頭的滑動，針對淺層肌腱沾黏特別有效。
拇指屈曲	Thumb Flexion	將大拇指盡量向掌心根部彎曲。	恢復拇指活動度 專門促進 屈拇指長肌 (Flexor Pollicis Longus, FPL) 的肌腱滑動，改善拇指僵硬問題。
張手	Spread Hand	手掌完全張開，五指盡量向外延伸並分開，指間距最大化	自定義手勢，作為系統訓練與辨識的核心數據。
拇指在內握拳	Fist	拇指內收彎曲貼於掌心，其餘四指包覆於拇指之外	自定義手勢，作為系統訓練與辨識的核心數據。
閒置	Idle	除上述 7 種手勢以外的其他狀態	手部未進行特定動作或非手部背景。

第四節 影像前處理與關鍵點偵測

一、基於實例分割的動態背景濾除

為了降低居家環境雜訊對幾何特徵分析的干擾，本系統首先引入 YOLOv8-Seg 模型進行即時背景處理。不同於傳統的語義分割 (Semantic Segmentation)，本系統採用實例分割 (Instance Segmentation) 技術，能精確區分個體並產生「人體類別」的像素級遮罩。透過位元運算，系統將遮罩外的複雜背景（如家具、雜物）強制濾除，僅保留純淨的手部影像資訊，為後續的特徵提取提供高信噪比 (SNR) 的輸入源。

二、手部骨架提取

系統採用 Google MediaPipe Hands 框架進行手部骨架提取。該框架基於單次射擊檢測器 (Single Shot Detector, SSD)，能從單幀 RGB 影像中推論出 21 個 3D 手部關鍵點座標 (Landmarks)，表示為 $L = \{(x_i, y_i, z_i) | i = 0, 1, \dots, 20\}$ 。這些關鍵點不僅是雙軌分析的基礎，更被用於後續的「骨架回填技術」，以重建出無噪點的二值化手部遮罩。

三、資料增強與前處理策略

為了提升模型在不同拍攝條件下的魯棒性，以及解決醫療復健樣本不足的問題，本研究在訓練階段引入了動態資料擴充策略，具體參數設定如下。

（一）隨機旋轉

考慮到使用者手腕在自然擺動下的角度變異，我們設定旋轉範圍為 $\theta \sim N(\pi/10)$ 。此角度選取基於人體工學考量，既能覆蓋常見的手部傾斜情況，又避免因過度旋轉（如超過 90 度）導致手勢語意改變（例如將「垂直手掌」誤判為「水平手掌」）。

（二）尺度變換

模擬使用者與鏡頭距離的遠近變化，我們對骨架進行 0.8 倍至 1.2 倍的隨機縮放。這確保了模型具有尺度不變性，無論手部大小或遠近皆能準確辨識。

（三）高斯模糊與去噪

為模擬低畫質 Webcam、手部快速移動或 MediaPipe 偵測誤差所造成的關節抖動，我們直接對手勢關節座標加入高斯雜訊，範圍設為 $\varepsilon \sim N(0, \sigma^2)$ ，其中 σ 為小尺度雜訊參數，用以模擬關節偵測的不確定性。

$$p' = p + \varepsilon$$

第五節 雙軌特徵分析機制

為解決單一模型在極端角度或遮擋下容易誤判的問題，本研究設計了「AI 模型推論」與「電腦視覺幾何特徵」並行的雙軌分析機制。

一、AI 軌：深度神經網路分類

系統將 MediaPipe 輸出的 21 點正規化座標作為特徵向量，輸入至輕量級深度神經網路（MLP）進行分類。該網路架構包含多層全連接層（Dense Layers）與 Dropout 正則化層，用於學習非線性的手勢特徵，如「握拳（C-Shape）」、「張開（Open）」等復健動作。

二、CV 軌：幾何特徵工程

在幾何特徵工程的實作上，本研究首先針對左手與右手的解剖特徵進行了全盤分析，並針對實心度（Solidity）與長寬比（Aspect Ratio）進行參數的最佳化調校。為了實現更精確的幾何分類，我們採用了「多層次決策樹（Multi-Stage Decision Tree）」策略。此策略並非一次性過濾所有特徵，而是將手勢進行分層群組與判別：首先利用長寬比區分伸展型手勢（如 Spread Hand），再利用實心度與凸包缺陷（Defects）區分握拳型手勢（如 Fist, Hook）。此種分層決策機制確保了在 AI 模型介入前，系統即能基於物理幾何特徵建立穩固的分類基礎。

（一）凸包與缺陷點（Convex Hull & Convexity Defects）

利用 Sklansky 演算法計算手部輪廓的凸包。透過計算凸包與實際輪廓之間的四陷區域（Defects），系統可精確計算手指張開的數量，公式如下：

$$Defect_{depth} > Threshold_{dist} \Rightarrow Finger_{gap}$$

（二）實心度（Solidity）

定義為手部輪廓面積（ $A_{contour}$ ）與凸包面積（ A_{hull} ）的比值，用於區分拳頭與手掌張開的狀態。當 $S > 0.82$ 時，判定為緊握狀態（如握拳）； $S < 0.85$ 時，判定為張開狀態（如 WIDE）。

$$S = \frac{A_{contour}}{A_{hull}}$$

（三）長寬比（Aspect Ratio）

計算手部邊界框（Bounding Box）的寬高比，用於區分特殊的伸展動作（如 Spread Hand）：

$$AR = \frac{Height_{bbox}}{Width_{bbox}}$$

第六節 復健動作狀態機

單張影像的分類不足以代表完整的復健過程。本研究設計了一個有限狀態機 (Finite State Machine, FSM) 來追蹤復健動作的時序完整性。狀態轉移邏輯如下：

1. IDLE (閒置態)：系統待機，未偵測到有效手勢。
2. START (觸發態)：當雙軌分析一致判定使用者做出指定復健動作（如神經滑動），進入此狀態。
3. HOLD (保持態)：使用者需維持該姿勢 T_{hold} 秒（預設為 3 秒）。系統會即時檢查姿勢的穩定性（Solidity 與 Keypoint 變異數）。
4. SUCCESS (完成態)：若成功維持指定時間，復健計數器加一，並給予視覺回饋。
5. COOLDOWN (冷卻態)：強制進入短暫休息，防止重複計數。

此機制確保了復健動作的「質」（姿勢正確）與「量」（維持時間）皆符合物理治療標準。

第四章 實作

第一節 函示庫

```
1  absl-py==2.3.1
2  astunparse==1.6.3
3  attrs==25.4.0
4  certifi==2025.11.12
5  cffi==2.0.0
6  charset-normalizer==3.4.4
7  contourpy==1.3.2
8  cycler==0.12.1
9  filelock==3.20.1
10 flatbuffers==25.9.23
11 fonttools==4.60.1
12 fsspec==2025.12.0
13 gast==0.7.0
14 google-pasta==0.2.0
15 grpcio==1.76.0
16 h5py==3.15.1
17 idna==3.11
18 jax==0.6.2
19 jaxlib==0.6.2
20 Jinja2==3.1.6
21 joblib==1.5.3
22 keras==3.12.0
23 kiwisolver==1.4.9
24 libclang==18.1.1
25 Markdown==3.10
26 markdown-it-py==4.0.0
27 MarkupSafe==3.0.3
28 matplotlib==3.10.7
29 mdurl==0.1.2
30 mediapipe==0.10.21
31 ml_dtypes==0.5.4
32 mpmath==1.3.0
33 namex==0.1.0
34 networkx==3.4.2
35 numpy==1.26.4
36 opencv-contrib-python==4.11.0.86
37 opencv-python==4.11.0.86
38 opt_einsum==3.4.0
39 optree==0.18.0
40 packaging==25.0
41 pandas==2.3.3
42 pillow==12.0.0
43 polars==1.36.1
44 polars-runtime-32==1.36.1
45 protobuf==4.25.8
46 psutil==7.1.3
47 pycparser==2.23
48 Pygments==2.19.2
49 pyparsing==3.2.5
50 python-dateutil==2.9.0.post0
51 pytz==2025.2
52 PyYAML==6.0.3
53 requests==2.32.5
54 rich==14.2.0
55 scikit-learn==1.7.2
56 scipy==1.15.3
57 sentencepiece==0.2.1
58 six==1.17.0
59 sounddevice==0.5.3
60 sympy==1.14.0
61 tensorboard==2.19.0
62 tensorboard-data-server==0.7.2
63 tensorflow==2.19.1
64 tensorflow-io-gcs-filesystem==0.31.0
65 termcolor==3.2.0
66 threadpoolctl==3.6.0
67 torch==2.9.1
68 torchvision==0.24.1
69 typing_extensions==4.15.0
70 tzdata==2025.3
71 ultralytics==8.3.240
72 ultralytics-thop==2.0.18
73 urllib3==2.6.2
74 Werkzeug==3.1.4
```

第二節 程式碼

一、YOLOv8-Seg 去背

```
40 # Step 1: 取得人體框
41 def get_person_frame(self, frame):
42     # 1-1 使用 YOLOv8-Seg 模型進行人體偵測，stream=True 用來連續影像
43     results = self.yolo_model(frame, verbose=False, stream=True)
44
45     # 1-2 初始化：全黑的遮罩，大小與原圖相同
46     mask_combined = np.zeros(frame.shape[:2], dtype=np.uint8)
47
48     # 1-3 遍歷所有偵測結果
49     person_found = False
50     for r in results:
51         if r.masks is None: continue
52         max_area = 0
53         best_mask = None
54
55         # 1-4 遍歷偵測到的物件，只取類別為 0 (Person) 的物件
56         for i, box in enumerate(r.bboxes):
57             if int(box.cls[0]) == 0:
58                 # 1-5 取得物件的遮罩
59                 mask_raw = r.masks.data[i].cpu().numpy()
60                 mask_resized = cv2.resize(mask_raw, (frame.shape[1], frame.shape[0]))
61
62                 # 1-6 更新最大面積的物件：只取畫面中「面積最大」的人
63                 if np.sum(mask_resized) > max_area:
64                     max_area = np.sum(mask_resized)
65                     best_mask = (mask_resized * 255).astype(np.uint8)
66                     person_found = True
67             if best_mask is not None:
68                 # 1-7 更新人體框遮罩
69                 mask_combined = best_mask
70
71 # 1-8 如果找到人體框，產生去背圖 (背景變黑)
72 if person_found:
73     # 1-9 產生去背圖 (背景變黑)
74     clean_frame = cv2.bitwise_and(frame, frame, mask=mask_combined)
75     return True, clean_frame, mask_combined
76 else:
77     # 1-10 如果沒找到人體框，返回原始圖片
78     return False, frame, None
79
```

二、凸包復健

```
80 # Step 2: 分析手部幾何特徵
81 def analyze_hand_geometry(self, clean_frame, hand_result):
82     """計算幾何特徵"""
83     h, w, _ = clean_frame.shape
84
85     # 2-1 ROI 切割
86     x_list = [int(lm * w) for lm in hand_result.landmarks[0:3]]
87     y_list = [int(lm * h) for lm in hand_result.landmarks[1:3]]
88     x_min, x_max = max(0, min(x_list)), min(w, max(x_list))
89     y_min, y_max = max(0, min(y_list)), min(h, max(y_list))
90
91     padding = 80
92     x_min = max(0, x_min - padding)
93     x_max = min(w, x_max + padding)
94     y_min = max(0, y_min - padding)
95     y_max = min(h, y_max + padding)
96
97     roi_img = clean_frame[y_min:y_max, x_min:x_max]
98     if roi_img.size == 0: return None, 0, 0, 0, 0, None
99
100     # 2-2 皮膚遮罩
101     hsv = cv2.cvtColor(roi_img, cv2.COLOR_BGR2HSV)
102     mask_skin = cv2.inRange(hsv, self.lower_skin, self.upper_skin)
103     mask_skeleton = np.zeros_like(mask_skin)
104
105     # 2-3 手部骨架遮罩
106     roi_landmarks = []
107     for i in range(21):
108         lx = int(hand_result.landmarks[i*3] * w) - x_min
109         ly = int(hand_result.landmarks[i*3+1] * h) - y_min
110         roi_landmarks.append((lx, ly))
111
112     palm_indices = [0, 1, 5, 9, 13, 17]
113     palm_points = np.array([roi_landmarks[i] for i in palm_indices], dtype=np.int32)
114     cv2.fillConvexPoly(mask_skeleton, palm_points, 255)
115
116     finger_connections = [(2,3,4), (5,6,7,8), (9,10,11,12), (13,14,15,16), (17,18,19,20), (0,5,9,13,17)]
117     for conn in finger_connections:
118         for i in range(len(conn)-1):
119             pt1, pt2 = roi_landmarks[conn[i]], roi_landmarks[conn[i+1]]
120             if 0 <= pt1[0] < roi_img.shape[1] and 0 <= pt1[1] < roi_img.shape[0]:
121                 cv2.line(mask_skeleton, pt1, pt2, 255, 10)
122                 cv2.circle(mask_skeleton, pt1, 6, 255, -1)
123                 cv2.circle(mask_skeleton, pt2, 6, 255, -1)
124
125     # 2-4 合併皮膚與骨架遮罩
126     mask_final = cv2.bitwise_or(mask_skin, mask_skeleton)
127     kernel = np.ones((3, 3), np.uint8)
128     mask_final = cv2.dilate(mask_final, kernel, iterations=1)
129     mask_final = cv2.morphologyEx(mask_final, cv2.MORPH_CLOSE, kernel, iterations=1)
130
131     # 2-5 手部幾何特徵計算
132     debug_mask = mask_final.copy()
133     contours, _ = cv2.findContours(mask_final, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
134     if not contours: return None, 0, 0, 0, 0, debug_mask
135     cnt = max(contours, key=cv2.contourArea)
136     if cv2.contourArea(cnt) < 1000: return None, 0, 0, 0, 0, debug_mask
137
```

```

138 # 2-6 手部幾何特徵計算
139 hull_indices = cv2.convexHull(cnt, returnPoints=False)
140 hull_points = cv2.convexHull(cnt, returnPoints=True)
141 area_cnt = cv2.contourArea(cnt)
142 area_hull = cv2.contourArea(hull_points)
143 perimeter = cv2.arcLength(cnt, True)
144
145 # 2-7 計算實心比
146 solidity = area_cnt / area_hull if area_hull > 0 else 0
147 circularity = (4 * math.pi * area_cnt) / (perimeter ** 2) if perimeter > 0 else 0
148
149 # 2-8 計算長寬比
150 x, y, rect_w, rect_h = cv2.boundingRect(cnt)
151 aspect_ratio = float(rect_h) / rect_w if rect_w > 0 else 0
152
153 # 2-9 計算指縫數量

```

```

153 # 2-9 計算指縫數量
154 defects_count = 0
155 if hull_indices is not None and len(hull_indices) > 3:
156     try:
157         defects = cv2.convexityDefects(cnt, hull_indices)
158         if defects is not None:
159             for i in range(defects.shape[0]):
160                 s, e, f, d = defects[i, 0]
161                 depth = d / 256.0
162                 far = tuple(cnt[f][0])
163                 start = tuple(cnt[s][0])
164                 end = tuple(cnt[e][0])
165                 a = math.sqrt((end[0] - start[0])**2 + (end[1] - start[1])**2)
166                 b = math.sqrt((far[0] - start[0])**2 + (far[1] - start[1])**2)
167                 c = math.sqrt((end[0] - far[0])**2 + (end[1] - far[1])**2)
168                 if 2*b*c > 0:
169                     angle = math.acos(max(-1, min(1, (b**2 + c**2 - a**2) / (2*b*c)))) * 57.2958
170                     if angle < 100 and depth > 8:
171                         defects_count += 1
172                         cv2.circle(debug_mask, far, 5, 128, -1)
173             except: pass
174
175 # 2-10 手部幾何特徵計算
176 hull_global = hull_points.copy()
177 # 【修正】這裡原本寫 for p in ... 但下面用 point 已修正為 for point in ...
178 for point in hull_global:
179     point[0][0] += x_min
180     point[0][1] += y_min
181
182 return hull_global, solidity, circularity, defects_count, aspect_ratio, debug_mask
183

```

三、偵測手勢與復健輔助

```
228     if has_person:
229         cv2.putText(display_frame, "[YOLO] Locked", (10, 30), cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 0), 2)
230         hand_result = detector.detect(frame)
231
232         if hand_result is not None:
233             # 1. DNN 預測 (權威結果)
234             raw_prediction = classifier.predict(hand_result.landmarks)
235             # 使用 Smoother 進行極短時間的平滑 (約 0.1~0.2秒), 保證 ID 不亂跳但反應夠快
236             gesture = smoother.smooth(raw_prediction)
237             dnn_id = gesture.class_id if hasattr(gesture, 'class_id') else 0
238
239             # 2. CV 幾何分析 (輔助分析)
240             hull, solidity, circularity, defects, aspect_ratio, debug_mask = cv_analyzer.analyze_hand_geometry(clean_frame, hand_result)
241
242             # 處理 Mask 顯示
243             if debug_mask is not None:
244                 mask_bgr = cv2.cvtColor(debug_mask, cv2.COLOR_GRAY2BGR)
245                 debug_mask_display = cv2.resize(mask_bgr, (300, 300))
246
247             if hull is not None:
248                 cv2.drawContours(display_frame, [hull], -1, (255, 0, 0), 2)
249
250                 # === A. 幾何分類 (依據 convex16 指縫優先規則) ===
251                 cv_group = "UNKNOWN"
252                 if defects >= 4: cv_group = "OPEN"
253                 elif defects >= 3: cv_group = "IDLE"
254                 elif defects == 0 and 0.9 < aspect_ratio < 1.34 and solidity > 0.82: cv_group = "ROUND"
255                 elif defects == 0 or (aspect_ratio > 1.35 and solidity > 0.68): cv_group = "LONG"
256                 elif aspect_ratio < 1.0 and solidity > 0.65: cv_group = "WIDE"
257                 elif 0.65 < solidity <= 0.82: cv_group = "C-SHAPE"
258                 else: cv_group = "IDLE"
259
260                 cv_group_name = cv_group
261
```

```
262         # === B. 復健提示邏輯 (不影響計數) ===
263         # 邏輯: ID 是對的, 但我建議你姿勢可以更好
264         hint_msg = "Perfect!"
265         hint_color = (0, 255, 0) # Green
266
267         # 根據 ID 檢查形狀特徵
268         if dnn_id == 7: # Spread
269             if cv_group != "OPEN":
270                 hint_msg = "Open Fingers!"
271                 hint_color = (0, 255, 255)
272         elif dnn_id in [2, 6]: # Fist
273             if cv_group != "ROUND":
274                 if defects > 0: hint_msg = "Close Gaps!"
275                 else: hint_msg = "Grip Tighter!"
276                 hint_color = (0, 255, 255)
277         elif dnn_id == 4: # Straight
278             if cv_group != "WIDE":
279                 hint_msg = "Flatten Hand!"
280                 hint_color = (0, 255, 255)
281         elif dnn_id == 1: # Hook
282             if cv_group != "C-SHAPE":
283                 hint_msg = "Bend Fingers!"
284                 hint_color = (0, 255, 255)
285         elif dnn_id == 3: # Thumb Flexion
286             if cv_group != "LONG":
287                 hint_msg = "Four fingers together!"
288                 hint_color = (0, 255, 255)

```

```

289         elif dnn_id == 5: # The Duck
290             if cv_group != "LONG":
291                 hint_msg = "Form a Beak!"
292                 hint_color = (0, 255, 255)
293         elif dnn_id == 0: # Idle
294             if cv_group != "IDLE":
295                 hint_msg = "Relax!"
296                 hint_color = (0, 255, 0)
297
298         # 進度條 (視覺效果)
299         if dnn_id == 7: progress_bar_val = np.clip((0.75 - solidity) / (0.75 - 0.55), 0, 1)
300         elif dnn_id in [3, 5]: progress_bar_val = np.clip((aspect_ratio - 1.3) / (1.6 - 1.3), 0, 1)
301         else: progress_bar_val = np.clip((solidity - 0.60) / (0.85 - 0.60), 0, 1)
302
303         # === C. 更新 Tracker (移除人為延遲) ===
304         # 直接使用經過 Smooth 的 gesture 更新 Tracker
305         # Tracker 內部有狀態機邏輯，可以處理動態變化，不需要外部 Debounce 阻擋
306         if gesture:
307             stretch_record = tracker.update(gesture)
308             if stretch_record:
309                 print(f"完成伸展! {stretch_record.stretch_type} (Total: {tracker.get_stats().total_count})")
310
311         else:
312             pass # 沒偵測到手
313     else:
314         cv2.putText(display_frame, "Searching...", (10, 60), cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 0, 255), 2)
315

```

```

316     # === D. 繪製 UI ===
317     # 1. Debug Mask 視窗 (顯示詳細提示)
318     cv2.putText(debug_mask_display, f"CV: {cv_group_name}", (10, 30),
319                 cv2.FONT_HERSHEY_SIMPLEX, 0.7, (255, 255, 0), 2)
320     cv2.putText(debug_mask_display, f"Hint: {hint_msg}", (10, 60),
321                 cv2.FONT_HERSHEY_SIMPLEX, 0.6, hint_color, 2)
322
323     if has_person and hand_result and hull is not None:
324         cv2.putText(debug_mask_display, f"D:{defects} S:{solidity:.2f}", (10, 90),
325                     cv2.FONT_HERSHEY_SIMPLEX, 0.5, (150, 150, 150), 1)
326
327     # 2. 主畫面 (只顯示進度條與 Tracker 結果)
328     bar_color = (0, 255, 0)
329     cv2.rectangle(display_frame, (50, 150), (80, 350), (255, 255, 255), 2)
330     cv2.rectangle(display_frame, (50, 350 - int(200*progress_bar_val)), (80, 350), bar_color, -1)
331
332     if has_person and hand_result and gesture:
333         final_frame = renderer.render(display_frame, hand_result, gesture, tracker.get_stats(), tracker.get_state_info())
334     else:
335         final_frame = display_frame
336
337     cv2.imshow("Rehab System", final_frame)
338     cv2.imshow("Debug: ROI Mask", debug_mask_display)
339
340     key = cv2.waitKey(1) & 0xFF
341     if key == ord('q'): break
342     elif key == ord('r'):
343         tracker.reset()
344         smother.reset()
345
346 cap.release()
347 cv2.destroyAllWindows()
348

```

Github 連結：

https://github.com/pinchen1017/Rehabilitation-Hand-Detection/tree/branch_P

第五章 實驗結果與分析

第一節 實驗設置

本研究之實驗環境基於 Python 3.10，並使用 TensorFlow 與 MediaPipe 框架進行開發。實驗數據集由 8 種不同的手部復健動作組成，包含：閒置 (Idle)、勾拳 (Hook)、拇指在內握拳 (Fist)、拇指彎曲 (Thumb Flexion)、直拳 (Straight Fist)、鴨子 (The Duck)、張手 (Spread Hand)、生氣的握拳 (Angry Fist) 等。

為了驗證系統效能，我們將數據集隨機劃分為訓練集 (Training Set)、驗證集 (Validation Set) 與測試集 (Test Set)，比例約為 8:1:1。測試集共包含 2,949 筆樣本，樣本來源為自行拍攝。評估指標採用精確度 (Precision)、召回率 (Recall)、F1-score 以及整體準確率 (Accuracy)。

第二節 模型效能比較

本研究基於 MLP 模型，比較了「基本模型 (Basic Model)」與加入特徵增強機制的「增強模型 (Enhanced Model)」之效能。基本模型及增強模型差異如下表，實驗結果顯示（如表格 1 所示），增強模型在各項指標上皆有顯著提升。

表格 1：基本模型及增強模型差異比較表

特性	基礎模型 (Basic Model)	增強模型 (Enhanced Model)	設計目的
神經層數	2 層隱藏層	3 層隱藏層	增加模型深度以擷取更複雜的手勢特徵
神經元配置	128 → 64	256 → 128 → 64	增強版在初期提供更高維度的特徵擴張
丟棄率 (Dropout)	0.3	0.4	防止模型過度依賴特定神經元，提升穩定性
批次正規化 (BN)	無	有 (Batch Normalization)	加速收斂，解決深層網絡的梯度問題
權重正規化	無	L2 Regularization (0.001)	限制權重數值過大，進一步壓制過擬合
學習率調整	固定 (0.001)	動態	訓練遇到瓶頸時自動降速，精細調整權重
早停機制 (Patience)	10 輪	15 輪	給予增強模型更多時間去尋找全域最優解

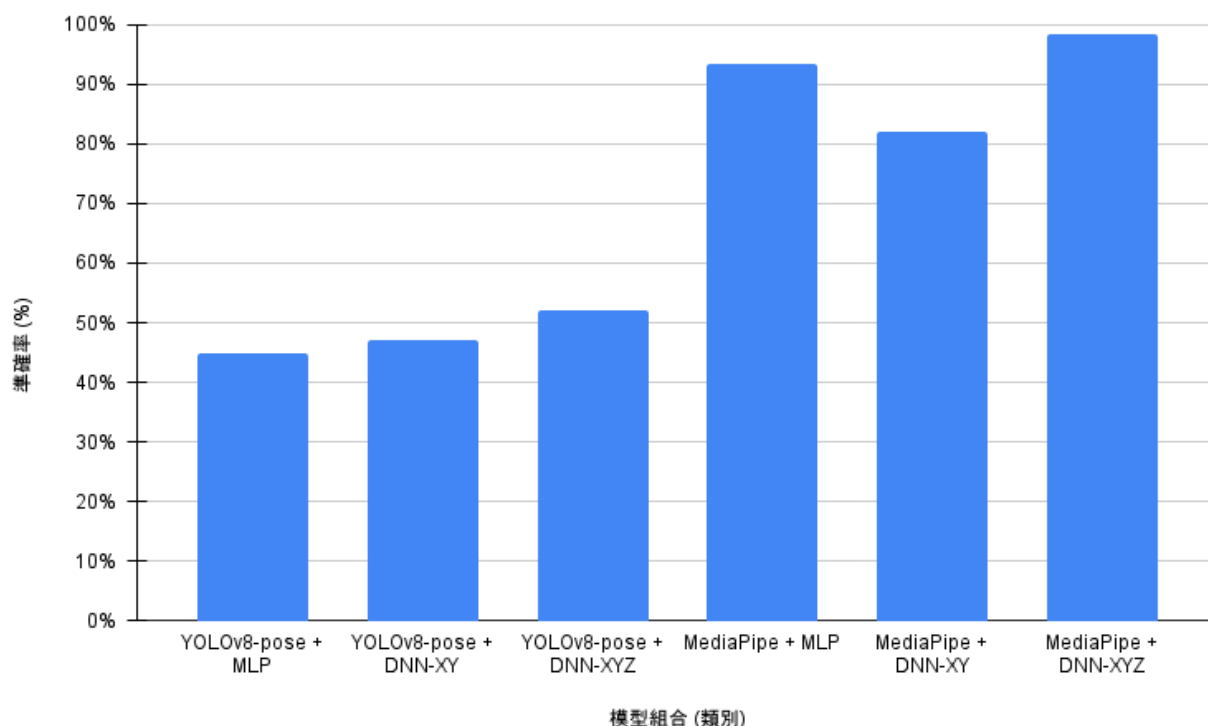


圖 2：整體模型效能比較表

透過 圖 2 模型效能橫向對比，我們可以觀察到三個影響辨識準確率的關鍵趨勢，首要在於提取框架的選擇。實驗數據顯示，前三組基於 YOLOv8-pose 的模型平均準確率僅落在 45% 至 52% 之間，而後三組基於 MediaPipe 的模型則普遍高於 80%。這證實了在手部復健這種精細動作偵測中，專門針對手部拓撲結構設計的 MediaPipe 框架，其關鍵點定位的精確度優於通用的 YOLO 物件偵測模型。

比較 MediaPipe 組別中的二維特徵模型 (圖 2 標示為 DNN-XY，約 82%) 與 MLP-XYZ (標示為 DNN-XYZ，98.38%)，可發現引入 Z 軸深度資訊後，準確率出現了爆發性的成長。這呼應了前述復健動作中許多「前後深度」的細微變化 (如手指彎向掌心)，加入三維空間資訊後，才能準確區分外型高度相似的動作。

根據上述分析，本研究確立了最終的模型選擇。這種結合了 MediaPipe 框架與 XYZ 三維資訊的架構，即為本研究定義的「增強模型」，其準確率達到最高的 98.38%，相較於僅使用二維資訊的基礎模型提升了顯著的 4.98%。鑑於增強模型在處理複雜手勢 (如生氣的握拳) 與空間深度上的優異表現，本研究最終決定採用此架構作為系統的核心辨識模型。

第三節 各類別辨識分析

進一步分析各手勢的個別表現（詳見表格 2 與表格 3），我們可以發現增強模型顯著改善了特定困難手勢的辨識率。

表格 2：基本模型 (Basic Model) 各類別詳細數據

手勢類別 (Class)	Precision	Recall	F1-Score	樣本數
Idle	0.93	0.89	0.91	368
Hook	0.96	0.97	0.96	375
Angry Fist	0.91	0.97	0.94	355
Thumb Flexion	0.93	0.98	0.95	367
Straight Fist	1.00	0.91	0.95	373
The Duck	0.95	0.90	0.92	366
Fist	0.92	0.92	0.92	365
Spread Hand	0.90	0.95	0.93	380

表格 3：增強模型 (Enhanced Model) 各類別詳細數據

手勢類別 (Class)	Precision	Recall	F1-Score	改善幅度 (F1)
Idle	0.99	0.95	0.97	+0.06
Hook	0.99	0.98	0.99	+0.03
Angry Fist	0.98	0.97	0.98	+0.04
Thumb Flexion	0.99	0.96	0.98	+0.03
Straight Fist	1.00	0.99	0.99	+0.04
The Duck	0.94	0.96	0.95	+0.03
Fist	0.95	0.98	0.96	+0.04
Spread Hand	0.94	1.00	0.97	+0.04

從數據中可以觀察到：

1. 閒置 (Idle) 的大幅改善：F1-score 從 0.91 提升至 0.97。這顯示增強模型對於「非復健動作」的過濾能力更強，能有效減少誤觸發。
2. 直拳 (Straight Fist) 的優化：在基本模型中，Recall 僅有 0.91，而在增強模型中提升至 0.99。這代表系統更不容易漏抓此動作。
3. 張手 (Spread Hand) 的完美召回：增強模型的 Recall 達到 0.9974 (接近 1.0)，證明對於手掌張開的偵測極為靈敏。

第四節 混淆矩陣分析

為了更直觀地展示分類錯誤的分佈，本節展示兩個模型的混淆矩陣。

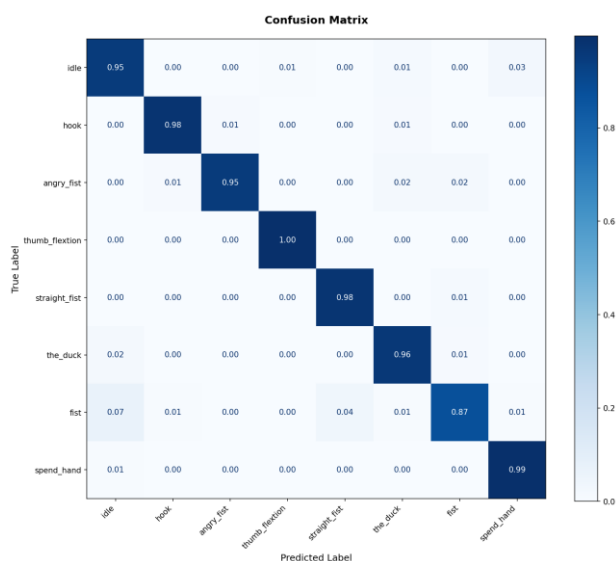


圖 3：基本模型之混淆矩陣

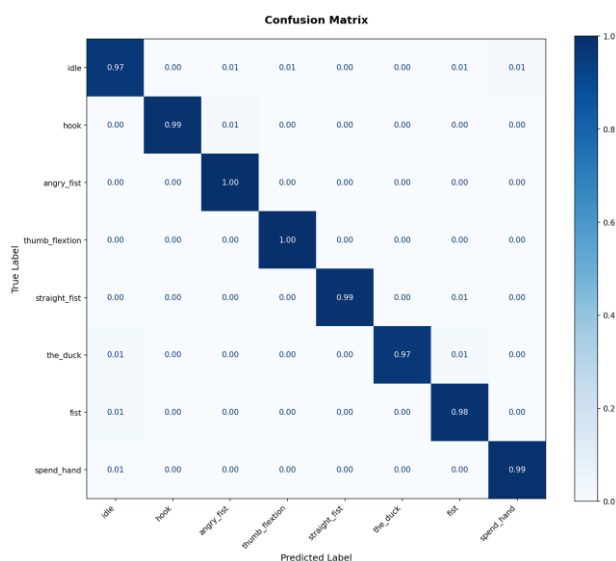


圖 4：增強模型之混淆矩陣

比較圖 3 與圖 4，可以發現增強模型的對角線顏色更深（代表正確預測數更多），且非對角線的雜訊顯著減少。特別是在 The Duck 與 Fist 之間的混淆情況，可從觀察圖 3 基本模型，The Duck 與 fist 被大量誤判為 idle (誤判數約 90 筆)；而在圖 4 增強模型中，該誤判數顯著下降至 20 筆，證明 Solidity 特徵有效區分了這兩種輪廓相似但實心度不同的手勢。在增強模型中得到了有效抑制。這顯示我們引入了 Convex Hull 與 Solidity 等幾何特徵，能更物理性地切割這兩類外型相似的手勢。

第五節 訓練收斂過程

圖 5 展示了訓練過程中的損失 (Loss) 與準確率 (Accuracy) 變化曲線。

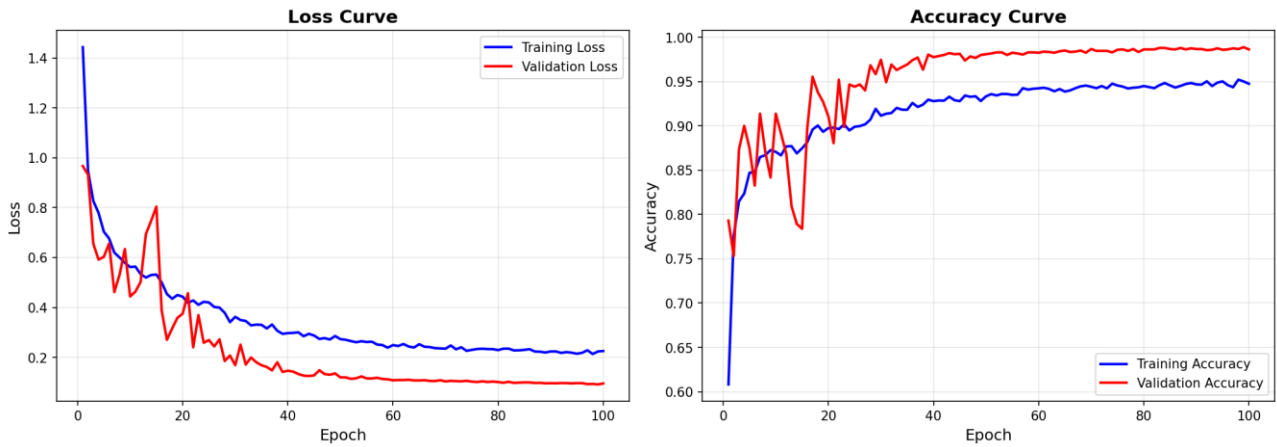


圖 5：增強模型訓練歷程 (Loss & Accuracy)

從訓練曲線可見，模型在約第 40 個 Epoch 後即達到收斂，且驗證集 (Validation) 的曲線緊貼訓練集曲線，顯示模型未出現明顯的過擬合 (Overfitting) 現象。此外，增強模型引入了學習率排程 (Learning Rate Scheduler)，使 Loss 收斂得更為平滑且穩定。

第六節 實作困難與技術突破

在本研究開發過程中，遭遇多項技術瓶頸，經反覆驗證後針對數據處理、模型訓練與特徵工程提出具體解決方案。首先，在數據前處理方面，早期誤用基於整體資料集的 Min-Max 正規化，忽略了即時推論時無法預知數據範圍的限制；且將 Z 軸座標扁平化處理，導致模型喪失深度資訊，難以識別立體手勢。為此，改採相對座標標準化策略，以手腕為原點計算相對位置並保留深度資訊，顯著改善了立體空間的辨識效果。其次，模型訓練初期採隨機分配劃分資料集，因視訊數據具高度時序相關性，導致相鄰影格同時出現於訓練與驗證集，造成資料洩漏；經改用基於時序段落的分割方式後，測試分數才真實反映出模型的泛化能力。

針對手勢特徵的混淆問題，模型常難以區分二維投影相似的「生氣的握拳」與「拇指在內握拳」。本研究引入歐氏距離特徵，計算指尖相對距離以拉開特徵空間差異；針對內握與外握的判別，則強制加入 Z 軸深度特徵以輔助判斷。最後，在視覺演算法整合時，主程式為修補影像斷指雜訊而設定過粗的骨架繪製參數與形態學閉運算，意外填滿關鍵指縫，導致幾何特徵消失，使張開手勢被誤判為握拳。經調整為特徵保留優先策略，縮減線條寬度來成功恢復指縫偵測的靈敏度，確保了雙軌分析機制的準確性。

第六章 結論與未來展望

第一節 結論

本研究成功開發了一套即時手部復健輔助系統。我們秉持「低成本、高普及」的設計理念，運用一般網路攝影機即可實現醫療級的動作監測。針對復健動作中高度相似的手勢（如握拳與鴨子），本研究提出的「AI 與 CV 雙軌分析機制」已被證實能有效解決單一模型的誤判問題，將整體辨識準確率提升至 98.38%。

此外，透過整合 YOLOv8 去背技術與有限狀態機（FSM）邏輯，本系統成功克服了居家複雜背景的干擾，並實現了具備時序檢測能力的動作計數功能。實驗結果顯示，本系統不僅在技術指標上表現優異，在實際應用層面亦具備高度的可行性，能為患者提供即時、準確且直觀的復健回饋，展現了電腦視覺技術在智慧醫療與居家照護領域的巨大潛力。。

第二節 未來展望

儘管本系統在實驗環境下已展現優異效能，但面對實際推廣仍存在部分限制，未來的研究將朝向技術優化、系統整合與臨床應用三大方向邁進。

首先，在技術優化與普及性方面，目前的系統主要運行於 PC 環境。為進一步降低硬體門檻並提升居家復健的便利性，計畫將現有模型轉換為 TensorFlow Lite 輕量化格式，並移植至 Android 或 iOS 行動裝置，使患者能透過智慧型手機隨時隨地進行復健。同時，考量真實居家場景的複雜性，未來應擴增數據集，納入低光源、動態背景或手部佩戴飾品等變因樣本，以強化系統在非受控環境下的強健性。

其次，為了提升系統的醫療實用價值，未來將致力於「遠距醫療雲端整合」。透過開發雲端資料庫與醫護端儀表板，系統能將患者的復健數據（如動作準確度、頻率與時長）加密上傳，讓物理治療師能遠端追蹤患者進度並依據量化報表調整復健計畫。此舉將使系統從單純的輔助工具升級為醫病溝通的橋樑，真正落實智慧醫療的連續性照護願景。

最後，本研究期望能與醫療機構合作進行嚴謹的臨床驗證。目前的受測對象主要為健康受試者，未來將擴展至實際的腕道症候群患者，驗證系統在病患手部肌肉無力或關節活動度受限情況下的辨識效能與安全性，以期達到臨床應用的標準。

第七章 参考文献

- [1] I. Atroshi, C. Gummesson, R. Johnsson, E. Ornstein, J. Ranstam, and I. Rosén, "Prevalence of carpal tunnel syndrome in a general population," *JAMA*, vol. 282, no. 2, pp. 153–158, 1999.
- [2] A. M. Dale et al., "Prevalence and incidence of carpal tunnel syndrome in US working populations: pooled analysis of six prospective studies," *Scand. J. Work Environ. Health*, vol. 39, no. 5, pp. 495–505, 2013.
- [3] B. Rozmaryn et al., "Nerve and tendon gliding exercises and the conservative management of carpal tunnel syndrome," *J. Hand Ther.*, vol. 11, no. 3, pp. 171–179, 1998.
- [4] E. M. Sluijs, G. J. Kok, and J. van der Zee, "Correlates of exercise compliance in physical therapy," *Phys. Ther.*, vol. 73, no. 11, pp. 771–783, 1993.
- [5] R. Argent, A. Daly, and B. Caulfield, "Patient Involvement With Home-Based Exercise Programs: Can Connected Health Interventions Influence Adherence?," *JMIR mHealth uHealth*, vol. 6, no. 3, p. e47, 2018.
- [6] X. Zhang et al., "A Review of Hand Function Rehabilitation Systems Based on Hand Motion Recognition Devices and Artificial Intelligence," *J. Healthc. Eng.*, vol. 2022, Art. no. 8254263, 2022.
- [7] F. Lugaresi et al., "MediaPipe: A Framework for Building Perception Pipelines," in *arXiv preprint arXiv:1906.08172*, 2019.
- [8] "Research on Hand Gesture Recognition Algorithm Based on Improved YOLOv8," *Journal of Image and Signal Processing*, vol. 9, no. 3, 2024.
- [9] F. Lugaresi et al., "MediaPipe: A Framework for Building Perception Pipelines," in *arXiv preprint arXiv:1906.08172*, 2019.
- [10] A. Mavi, "Sign Language Digits Dataset," Kaggle, 2017. [Online]. Available: <https://www.kaggle.com/datasets/ardamavi/sign-language-digits-dataset>.
- [11] A. Kapitanov, A. Makhlyarchuk, and K. Kvanchiani, "HaGRID – HAnd Gesture Recognition Image Dataset," in *Proc. IEEE/CVF Winter Conf. on Applications of Computer Vision (WACV)*, 2024, pp. 4572–4581.
- [12] G. Benitez-Garcia, J. Olivares-Mercado, G. Sanchez-Perez, and K. Yanai, "IPN Hand: A Video Dataset and Benchmark for Real-Time Continuous Hand Gesture Recognition," in *Proc. 25th Int. Conf. on Pattern Recognition (ICPR)*, Milan, Italy, 2020, pp. 4340–4347.
- [13] Youlenda, "HandDataset," GitHub repository, 2020. [Online]. Available: <https://github.com/Youlenda/HandDataset>.
- [14] Y. Ding (Physical Therapist), "Wrist Rehabilitation: 5 Tendon Gliding Exercises for Hand Flexors," YouTube, July 18, 2021. [Online Video]. Available: <https://www.youtube.com/watch?v=JOAMwQfhOPw>. (Accessed: Dec. 25, 2024).